

Web Basic

ES6 문법과 내장 객체

내장 객체

내장 객체 1

- 자바스크립트는 여러 용도에 활용하는 객체를 내장하고 있다.
- 숫자 다루기, 문자 다루기, 날짜 다루기, JSON 객체 다루기 등에 유용한 객체를 제공한다.
- 핵심 내장 객체들의 기능을 이해하면, 실제 프로젝트에서 유용하게 활용할 수 있다.

```
const targetURL = "https://www.naver.com";  
const windowSize = `height=${window.innerHeight}, width=${window.innerWidth}`;  
window.open(targetURL, "Target", windowSize);
```

DOM document를 포함하는 브라우저 창을 나타내는 객체.

현재 창의 정보를 얻거나, 창을 조작할 수 있다.

(브라우저 환경에서는 window 객체가 전역 객체이다.)



```
function printDocumentInfo() {  
  console.log("문서 URL : ", document.URL);  
  console.log("문서 타이틀 : ", document.title);  
  console.log("모든 노드 : ", document.querySelectorAll("*"));  
}
```

브라우저에 로드된 웹 페이지에 대한 객체.

문서의 title, URL 등의 정보를 얻을 수 있고, 요소 생성, 검색 등의 기능을 제공한다.



Number

```
function changeToUsd(krw) {  
  const rate = 1046;  
  return (krw / rate).toFixed(2);  
}  
  
const krw = 1000000;  
console.log(changeToUsd(krw)); // '956.02'
```

자바스크립트의 number 자료형을 감싸는 객체로,
유의미한 상수값, 숫자를 변환하는 메서드 등을 제공한다.



Number

```
function changeToUsd(krw) {  
  const rate = 1046;  
  return (krw / rate).toFixed(2);  
}  
  
const krw = 1000000;  
console.log(changeToUsd(krw)); // '956.02'
```

`Number.toFixed` 메서드는 숫자의 소숫점 자릿수를 제어한다.
위 예시에서 반환된 값은 소숫점 둘째 자리까지 반올림한 문자열이다.



```
function getMaxDiff(nums) {  
  return Math.max(...nums) - Math.min(...nums);  
}  
  
getMaxDiff([-1, -4, -7, 11]); // 18
```

기본적인 수학 연산 메서드, 상수를 다루는 객체.


```
function getMaxDiff(nums) {  
  return Math.max(...nums) - Math.min(...nums);  
}
```

*Spread syntax

```
getMaxDiff([-1, -4, -7, 11]); // 18
```

‘Math.max’, ‘Math.min’은 개별 숫자를 인자로 받아 각각 최대, 최솟값을 리턴한다.



```
function getRandomNumberInRange(min, max) {  
  return Math.floor(Math.random() * (max - min + 1)) + min;  
}  
  
console.log(getRandomNumberInRange(50, 100));
```

‘Math.random’은 0에서 1 사이의 float number을 랜덤으로 구한다.

‘Math.floor’는 소숫점 이하 숫자를 버린다.

내장 객체 2



Date

```
function isWeekend(today) {  
  let day = today.getDay();  
  return day === 0 || day === 6;  
}  
  
console.log(isWeekend(new Date("2024/9/14")));
```

특정 시점의 날짜를 표시하기 위한 객체.

날짜와 관련된 작업을 하기 위한 여러 메서드를 포함한다.


```
function isWeekend(today) {  
  let day = today.getDay();  
  return day === 0 || day === 6;  
}  
  
console.log(isWeekend(new Date("2024/9/14")));
```

`Date.getDay`는 요일을 0(일요일)부터 6(토요일)로 반환한다.

이 외에 년도, 월, 일, 시, 분, 초, 밀리초 등을 구할 수 있다.

```
function addDays(date, days) {  
  date.setDate(date.getDate() + days);  
  return date.toString();  
}  
  
addDays(new Date("2024/9/22"), 100); // 'Tue Dec 31 2024'
```

`setDate` 등의 메서드로 시간을 설정한다.

`toString` 메서드는 특정 포맷의 날짜로 문자열을 반환한다.

```
function timeDiff(date1, date2) {  
  return date2.getTime() - date1.getTime();  
}  
  
let dayTime = 60 * 24 * 60 * 1000;  
function fromNow(date) {  
  let diff = timeDiff(date, new Date());  
  return `${Math.floor(diff / dayTime)} days ago...`;  
}  
  
fromNow(new Date("2024/03/23")); // 예시: '61 days ago...'
```

‘getTime’ 메서드는 1970.01.01 시점부터 흐른 시간을 밀리초 단위로 반환한다.

```
"hello".toUpperCase();           // 'HELLO'
"Daniel, Kim, SW".split(',');      // [ 'Daniel', 'Kim', 'SW' ]
"Daniel, Kim, SW".replace(',', ' '); // 'Daniel Kim, SW'
"Daniel, Kim, SW".includes("Kim"); // true
" Daniel, Kim, SW ".trim();         // 'Daniel, Kim, SW'
"Daniel, Kim, SW".indexOf("Kim");  // 7
```

문자열을 조작하기 위한 여러 메서드를 포함한다.



JSON

```
// 객체 직렬화
```

```
JSON.stringify({ name: "Daniel", age : 12}) // '{"name":"Daniel","age":12}'
```

```
// 역직렬화
```

```
JSON.parse('{"name":"Daniel","age":12}') // { name: 'Daniel', age: 12 }
```

자바스크립트 객체를 직렬화 / 역직렬화 하는데 사용되는 객체.

데이터 통신 과정에서 자주 사용된다.

직렬화: 객체 데이터를 다른 시스템에서도 사용할 수 있도록 바이트 스트림 형태의 연속적인 데이터로 변환하는 포맷 기술

역직렬화: 직렬화 된 데이터를 다시 원래의 객체 형태로 되돌리는 것